



## 1. Descripción general de la solución

El objetivo del trabajo fue extender un controlador POX básico para implementar un NAT con traducción por puertos (PAT) sobre una red emulada con Mininet. La solución se desarrolló a partir del controlador provisto por la cátedra, agregando tres componentes principales:

1. **Manejo de ARP dinámico:** el controlador responde ARP Requests dirigidos a las interfaces del router y genera ARP Requests cuando necesita resolver direcciones MAC desconocidas.
2. **NAT por puertos (PAT):** el controlador traduce direcciones IP y puertos de transporte para permitir que múltiples hosts privados compartan una única dirección IP pública.
3. **Instalación de flujos OpenFlow:** una vez establecida la traducción y resuelta la dirección MAC destino, se instalan reglas en el switch para que el forwarding posterior no requiera intervención del controlador.

El sistema final permite que los hosts de la red privada (192.168.1.0/24) se comuniquen con la red pública (200.0.0.0/24) de forma transparente, soportando tráfico TCP, UDP e ICMP (ping).

## 2. Topología de red

Se partió de la topología base provista y se la extendió agregando hosts adicionales en la red privada. La topología final cuenta con:

- **h1:** host público (200.0.0.1/24), conectado al puerto 1 del switch.
- **h2:** host privado (192.168.1.2/24), conectado al puerto 2.
- **h3:** host privado (192.168.1.3/24), conectado al puerto 3.
- **h4:** host privado (192.168.1.4/24), conectado al puerto 4.

El switch **s1** actúa como dispositivo de capa 3, con dos interfaces lógicas:

| Interfaz | IP            | MAC               |
|----------|---------------|-------------------|
| Pública  | 200.0.0.254   | 00:00:00:aa:aa:aa |
| Privada  | 192.168.1.254 | 00:00:00:bb:bb:bb |

Cuadro 1: Interfaces del router/switch

Se eliminaron todas las entradas ARP estáticas que existían en la topología original, dejando que la resolución de direcciones MAC se realice íntegramente mediante el protocolo ARP implementado en el controlador.

## 3. Manejo de ARP

El controlador implementa manejo de ARP dinámico, necesario para que el NAT pueda resolver direcciones MAC en ambas redes sin depender de entradas estáticas.

### 3.1. Tabla ARP dinámica

Se mantiene un diccionario `tabla_arp` que mapea `IPAddr` → `EthAddr`. Esta tabla se actualiza cada vez que se recibe cualquier paquete ARP (Request o Reply), aprendiendo la asociación IP-MAC del emisor:

```
1 self.tabla_arp[arp_pkt.protosrc] = arp_pkt.hwsrc
```

Listing 1: Aprendizaje ARP dinámico

### 3.2. ARP Reply (respuesta a ARP Requests)

Cuando un host envía un ARP Request preguntando por la MAC de alguna de las interfaces del router (192.168.1.254 o 200.0.0.254), el controlador genera un ARP Reply con la MAC correspondiente y lo envía de vuelta por el mismo puerto de ingreso. Los requests destinados a otras IPs son ignorados.

### 3.3. ARP Request (resolución de MACs desconocidas)

Cuando el controlador necesita reenviar un paquete IP pero no conoce la MAC destino, encola el paquete en `paquetes_esperando` y genera un ARP Request hacia la red correspondiente:

- Si el destino es un host público, el ARP Request sale por el puerto público con IP/MAC de la interfaz pública como origen.
- Si el destino es un host privado, el ARP Request sale por el puerto correspondiente (o por flood si no se conoce el puerto) con IP/MAC de la interfaz privada.

### 3.4. Procesamiento de ARP Replies

Al recibir un ARP Reply, el controlador:

1. Actualiza la tabla ARP con la asociación aprendida.
2. Desencola todos los paquetes que estaban esperando la resolución de esa dirección.
3. Reenvía cada paquete encolado con las MACs y el puerto de salida correctos.

## 4. Diseño de la tabla NAT

El NAT utiliza dos diccionarios complementarios que representan una tabla de traducción bidireccional:

### 4.1. Tabla saliente (`nat_saliente`)

| Clave (tupla)                                         | Valor      |
|-------------------------------------------------------|------------|
| (protocolo, IP_priv, puerto_priv, IP_dst, puerto_dst) | puerto_pub |

Cuadro 2: Estructura de la tabla NAT saliente

La clave identifica unívocamente una conexión desde la red privada. El valor es el puerto público asignado en la interfaz NAT.

### 4.2. Tabla entrante (`nat_entrante`)

| Clave (tupla)                               | Valor (tupla)          |
|---------------------------------------------|------------------------|
| (protocolo, puerto_pub, IP_dst, puerto_dst) | (IP_priv, puerto_priv) |

Cuadro 3: Estructura de la tabla NAT entrante

Esta tabla es la inversa de la anterior: dado un paquete entrante desde la red pública, permite determinar a qué host y puerto privado debe traducirse.

### 4.3. Asignación de puertos públicos

Los puertos públicos se asignan de forma secuencial a partir del puerto **50000**, dentro del rango de puertos efímeros. El algoritmo de asignación:

1. Toma el siguiente puerto candidato (**proximo\_puerto\_publico**).
2. Incrementa el contador de forma circular (vuelve a 50000 al superar 65535).
3. Verifica que la tupla (**protocolo, puerto, IP\_dst, puerto\_dst**) no exista en **nat\_entrante**.
4. Si está libre, lo asigna. Si no, prueba con el siguiente.
5. Si se agotan los 15536 puertos disponibles, se descarta el paquete.

Este diseño permite la reutilización de un mismo puerto público para conexiones con destinos distintos, ya que la clave incluye el destino, lo que evita colisiones.

### 4.4. Ejemplo de traducción

| Protocolo | IP privada  | Puerto priv. | IP pública  | Puerto púb. |
|-----------|-------------|--------------|-------------|-------------|
| TCP       | 192.168.1.2 | 12345        | 200.0.0.254 | 50000       |
| TCP       | 192.168.1.3 | 12345        | 200.0.0.254 | 50001       |
| UDP       | 192.168.1.2 | 54321        | 200.0.0.254 | 50002       |

Cuadro 4: Ejemplo conceptual de la tabla NAT

## 5. Estrategia de instalación de flujos

La instalación de flujos tiene como objetivo minimizar la intervención del controlador: una vez establecida la traducción para una conexión, el switch debe poder procesar los paquetes siguientes sin consultar al controlador.

### 5.1. Flujos instalados

Para cada nueva conexión TCP o UDP, se instalan **dos flujos** en el switch:

**Flujo saliente** (privado → público):

- **Match:** IP origen privada, IP destino pública, puertos origen/destino, protocolo, puerto de entrada.
- **Acciones:** reescribir IP origen a **PUBLIC\_IP**, reescribir puerto origen al puerto público asignado, cambiar MACs y enviar por el puerto público.

**Flujo entrante** (público → privado):

- **Match:** IP origen pública, IP destino = **PUBLIC\_IP**, puerto destino = puerto público asignado, protocolo, ingreso por puerto público.
- **Acciones:** reescribir IP destino a la IP privada original, reescribir puerto destino al puerto privado original, cambiar MACs y enviar al puerto del host privado.

## 5.2. Timeouts y limpieza de estado

Los flujos se instalan con un `idle_timeout` de **10 segundos**: si no se recibe tráfico para ese flujo durante 10 segundos, el switch lo elimina automáticamente y envía un mensaje `FlowRemoved` al controlador.

Al recibir un `FlowRemoved`, el controlador:

1. Identifica la entrada NAT correspondiente al flujo expirado.
2. Elimina tanto la entrada de `nat_saliente` como la de `nat_entrante`.
3. Libera el puerto público para su reutilización.

Esto se configura mediante el flag `OFPPF_SEND_FLOW_REM` al instalar el flujo.

## 5.3. ICMP

Los paquetes ICMP Echo Request/Reply se traducen correctamente (usando el campo `id` del ICMP como identificador de la conexión), pero **no se instalan flujos en el switch** para ellos. Cada paquete ICMP es procesado por el controlador. Esta decisión se tomó debido a que:

- El tráfico ICMP lo usamos para hacer ping entre hosts únicamente.
- La traducción de ICMP no usa puertos de transporte por lo que la instalación de flujos es distinta a TCP/UDP.

## 6. Decisiones de diseño

1. **Clave NAT con destino incluido:** la clave de la tabla saliente incluye la IP y puerto destino, lo que permite reutilizar el mismo puerto público para conexiones de un mismo host a destinos diferentes. Esto maximiza la eficiencia en el uso de puertos públicos.
2. **Cola de paquetes durante resolución ARP:** en lugar de descartar paquetes cuya MAC destino es desconocida, se los encola y se envía un ARP Request. Al recibir el ARP Reply, se procesan todos los paquetes encolados. Esto evita la pérdida del primer paquete de cada conexión.
3. **Un ARP Request por IP desconocida:** solo se envía un ARP Request la primera vez que se necesita resolver una IP. Si llegan más paquetes para esa misma IP antes de recibir el reply, se encolan sin generar requests adicionales.
4. **Mapeo IP → puerto del switch:** se mantiene un diccionario `ip_to_port` que registra el puerto físico del switch por el que se vio tráfico de cada IP. Esto permite dirigir paquetes entrantes al puerto correcto sin necesidad de flood.

## 7. Problemas encontrados y soluciones

1. **Pérdida del primer paquete por ARP no resuelto:**  
*Problema:* al enviar el primer paquete hacia un host cuya MAC es desconocida, el paquete se perdía porque no se podía armar la trama Ethernet.  
*Solución:* se implementó una cola de paquetes (`paquetes_esperando`) que almacena los paquetes mientras se resuelve la MAC mediante ARP. Al recibir el ARP Reply, se reenvían los paquetes encolados.

## 8. Pruebas realizadas

### 8.1. Conectividad básica

Se verificó la conectividad entre un host privado y el host público mediante `ping`:

```
1 mininet> h2 ping -c 4 h1
```

Listing 2: Prueba de ping básico

Se espera que los primeros paquetes tengan mayor latencia (por la resolución ARP) y los siguientes sean más rápidos (flujos instalados en el switch, para los protocolos en los que aplica).

### 8.2. Múltiples clientes simultáneos

Se probó la conectividad simultánea desde los tres hosts privados (h2, h3, h4) hacia el servidor público (h1):

```
1 mininet> h2 ping -c 4 h1 &  
2 mininet> h3 ping -c 4 h1 &  
3 mininet> h4 ping -c 4 h1
```

Listing 3: Prueba con múltiples clientes

Cada host obtiene un puerto público distinto para su traducción ICMP, verificando que el NAT maneja correctamente múltiples conexiones simultáneas sin colisiones.

### 8.3. Tráfico TCP

Se utilizó `iperf` para probar tráfico TCP:

```
1 # En h1 (servidor):  
2 mininet> h1 iperf -s &  
3  
4 # En h2 (cliente):  
5 mininet> h2 iperf -c 200.0.0.1 -t 5
```

Listing 4: Prueba TCP con iperf

### 8.4. Tráfico UDP

Se probó con `iperf` en modo UDP:

```
1 # En h1 (servidor):  
2 mininet> h1 iperf -s -u &  
3  
4 # En h2 (cliente):  
5 mininet> h2 iperf -c 200.0.0.1 -u -t 5
```

Listing 5: Prueba UDP con iperf

### 8.5. Verificación de flujos instalados

Se verificó que los flujos se instalen correctamente en el switch:

```
1 mininet> dpctl dump-flows
```

Listing 6: Inspección de flujos en el switch

## 8.6. Verificación con Wireshark/tcpdump

Se capturó tráfico en la interfaz del host público para verificar que los paquetes lleguen con la IP pública del NAT (200.0.0.254) y un puerto traducido:

```
1 mininet> h1 tcpdump -i h1-eth0 -n
```

Listing 7: Captura de tráfico

## 9. Conclusión

Se implementó exitosamente un NAT con traducción por puertos (PAT) como aplicación SDN sobre el framework POX. La solución cumple con todos los requisitos funcionales especificados:

- Múltiples hosts privados se comunican con la red pública compartiendo una única IP.
- Se soporta tráfico TCP, UDP e ICMP.
- La resolución de direcciones MAC se realiza dinámicamente mediante ARP, sin entradas estáticas.
- Se mantiene estado de las conexiones activas con limpieza automática por inactividad.
- La traducción opera correctamente en ambos sentidos (saliente y entrante).
- Los flujos instalados en el switch minimizan la intervención del controlador.

El desarrollo de este trabajo nos permitió comprender en profundidad el paradigma SDN y los beneficios de la separación entre plano de control y plano de datos. A su vez, nos permitió entender mejor funcionalidades de red complejas como NAT.